

PowerIO: Compiler infrastructure for power systems

Translating power system data formats with a shared, typed library

Samuel Talkington

Harvard SEAS

September 11, 2026, PowerUp 2026, Boulder

Workshop of the IEEE PES Task Force on Benchmarking Multiconductor OPF

- In PowerModels.jl, `parse_file("case118.m")` loads **JuMP and its whole dependency tree**.
- Users have asked for a **standalone parser** for years, in the PowerModels.jl issue tracker and on Julia discourse.
- Each project's own reader **fails on some real cases**, reproduced in PowerModels, pandapower, and PMD.

lanl-ansi/PowerModels.jl



#894 Move data parser to seperate package

 2 comments

 **noahrhodes** opened on November 3, 2023



github.com/lanl-ansi/PowerModels.jl, issue open since 2023



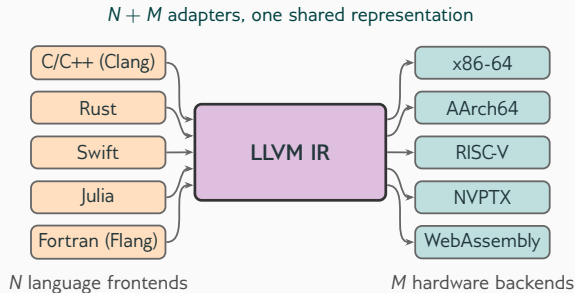
@frapac

there has long been a dream of @claytonpbarrows and I to have a "parser-only" package that could be used by the variety of power system tools that are out there in Julia. The challenge in the past has been, what data format would such a parser target? Maybe @npr has found a reasonable proposal for that challenge in this package.

C. Coffrin on discourse.julialang.org, Nov 2021

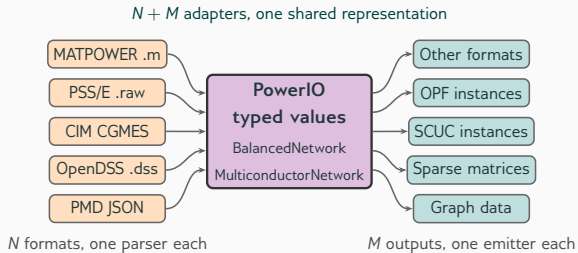
Sources: PowerModels.jl issue 894 (2023, open); Julia discourse 71722 (2021); PowerModelsDistribution.jl issue 398 (2022). Reproduced in [eigenergy/powerio evals](#).

- LLVM¹ began at Illinois in 2000 as **reusable compiler libraries** around one shared, typed representation.
- Language frontends write it; hardware backends read it. A new language or chip is **one new adapter**.
- Clang (C and C++), Rust, Swift, and Julia all compile through it.



¹C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation," CGO 2004. Dragon logo: llvm.org.

- A parse returns **typed values plus diagnostics**, not a dictionary.
- Two network types, balanced and multiconductor. Each format needs **one parser and one emitter**.
- From LLVM, PowerIO keeps a small core, diagnostics, and explicit transformations, and leaves out the low-level compiler machinery. **The data stays at the level of buses and lines.**



eigenergy/powerio documentation, "LLVM and MLIR lessons" and "Architecture," eigenergy.github.io/powerio.

Three rules

1. Same format in and out returns the **exact source bytes**.
2. A different format out keeps what fits and **reports every dropped field**.
3. Changing the network type takes an explicit call, **never a silent one**.

- 12 balanced and 3 multiconductor formats, including **OpenDSS** and **BMOPF draft JSON**.
- Callable from Rust, Python, Julia, C, and WebAssembly. 1,800+ tests and 14 fuzz targets.
- Parsing case9241pegase takes **10 ms**, against 0.79 s in PowerModels.jl.
- Used downstream by tellegen, ExaModelsPower, and BMOPFTools.

Licensed MIT or Apache-2.0. Timing: `evals/performance/RESULTS.md`, Rust and Julia bindings, one machine; the PowerModels.jl 0.21.6 parse also validates.

```
$ powerio convert ieee34Mod1.dss -to bmopf-json -o ieee34.bmopf.json
```

ieee34Mod1.dss and IEEELineCodes.DSS (excerpt)

```
New linecode.300 nphases=3 basefreq=60 units=kft
~ rmatrix = [0.253181818 | 0.039791667
             0.250719697 | ...]

New Line.L1 Phases=3 Bus1=800.1.2.3
             Bus2=802.1.2.3 LineCode=300 Length=2.58
             units=kft

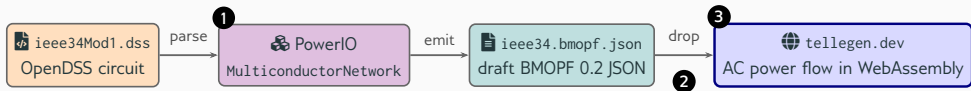
New Load.S860 Bus1=860 Phases=3 Conn=Wye Model=1
             kV= 24.900 kW= 60.0 kVAR= 48.0
```

Writing the circuit back to .dss and re-solving in OpenDSS agrees within 10^{-8} pu on IEEE 13, 34, and 123.

ieee34.bmopf.json (draft BMOPF 0.2)

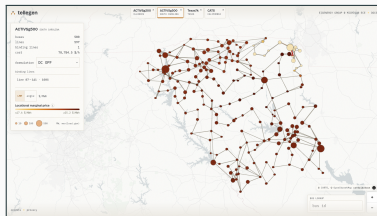
```
"line": { "L1": { "bus_from": "800",
                  "bus_to": "802", "length": 786.384,
                  "linecode": "300", ... } },
"linecode": { "300": { ...,
                    "R_series_1_1": 0.00083064..., ... } },
"load": { "S860": { "bus": "860", ...,
                    "model": "CONSTANT_POWER",
                    "p_nom": [20000.0, 20000.0, 20000.0], ... } }
```

Schema-valid, and reads back to the same network. 306 dropped fields are **reported**, e.g. vminpu and RegControl settings.



1. Convert with the command on the previous slide.
2. Drop `ieee34.bmpf.json` on `tellegen.dev`. PowerIO parses it **in the browser**.
3. Open **Studies**, choose **Run power flow**: terminal voltages, conductor currents, source power, losses. Multiconductor AC OPF is next.

tellegen.dev



tellegen.dev: case files are parsed and solved in the browser.

Source: [eigenergy/tellegen](https://eigenergy.github.io/tellegen/). Case parsing uses powerio; local case files never leave the browser.

Questions?

samuel_talkington@seas.harvard.edu



github.com/eigenergy/powerio



tellegen.dev



eigenergy.github.io/powerio

```
cargo add powerio  pip install powerio  Pkg.add("PowerIO")
```



eigenergy